In [4]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# 1. Load files
# ----------------------------

regional_files = {
    "Dushanbe": "DB Impact source hubs 2025- 2026 - Dushanbe ISH.csv",
    "Shugnan": "DB Impact source hubs 2025- 2026 - Shugnan ISH.csv",
    "Bokhtar": "DB Impact source hubs 2025- 2026 - Bokhtar ISH.csv",
    "Panjikent": "DB Impact source hubs 2025- 2026 - Panjikent ISH.csv"
}

employment_file = "DB Impact source hubs 2025- 2026 - Employment.csv"

all_students = []

for region, file in regional_files.items():
    temp = pd.read_csv(file)
    temp["region_clean"] = region
    all_students.append(temp)

students = pd.concat(all_students, ignore_index=True)
employment = pd.read_csv(employment_file)

# ----------------------------
# 2. Clean column names
# ----------------------------

def clean_columns(df):
    df.columns = (
        df.columns
        .str.strip()
        .str.lower()
        .str.replace("\n", " ", regex=False)
        .str.replace("  ", " ", regex=False)
    )
    return df

students = clean_columns(students)
employment = clean_columns(employment)

# ----------------------------
# 3. Rename useful columns
# ----------------------------

students = students.rename(columns={
    "full name": "student_name",
    "gender пол": "gender",
    "age возраст": "age",
    "courses received/ название курса": "course",
    "pre-test": "pre_test",
```

```
    "post-test": "post_test",
    "progress made": "progress",
    "amount earned (tjs)": "amount_earned"
})

employment = employment.rename(columns={
    "full name": "student_name",
    "courses received/ название курса": "course",
    "province name": "region",
    "organization employed": "organization_employed"
})

# ----------------------------
# 4. Basic cleaning
# ----------------------------

students["course"] = students["course"].astype(str).str.strip()
employment["course"] = employment["course"].astype(str).str.strip()

students["student_name"] = students["student_name"].astype(str).str.lower().
employment["student_name"] = employment["student_name"].astype(str).str.lowe

students = students[students["course"].notna()]
students = students[students["course"] != "nan"]

for col in ["pre_test", "post_test", "progress", "amount_earned", "age"]:
    if col in students.columns:
        students[col] = (
            students[col]
            .astype(str)
            .str.replace("%", "", regex=False)
            .str.replace(",", "", regex=False)
        )
        students[col] = pd.to_numeric(students[col], errors="coerce")

# ----------------------------
# 5. Add employment flag
# ----------------------------

employment["employed_flag"] = 1

students = students.merge(
    employment[["student_name", "employed_flag"]],
    on="student_name",
    how="left"
)

students["employed_flag"] = students["employed_flag"].fillna(0)

# ----------------------------
# 6. Course-level summary
# ----------------------------

course_summary = students.groupby("course").agg(
    students_count=("student_name", "count"),
    avg_pre_test=("pre_test", "mean"),
```

```
    avg_post_test=("post_test", "mean"),
    avg_progress=("progress", "mean"),
    employment_count=("employed_flag", "sum"),
    employment_rate=("employed_flag", "mean"),
    female_share=("gender", lambda x: (x.astype(str).str.lower().str.contair
    avg_age=("age", "mean")
).reset_index()

# ----------------------------
# 7. Add future demand scores
# ----------------------------
# These are initial assumptions based on WEF future skills trends.
# You can adjust these later.

future_demand_scores = {
    "Digital Literacy": 8,
    "Digital literacy": 8,
    "ICT": 8,
    "Java Script": 9,
    "Programming": 9,
    "Basics of Programming": 8,
    "Graphic Design": 6,
    "SMM": 6,
    "Web design": 7
}

course_summary["future_demand_score"] = course_summary["course"].map(future_

# ----------------------------
# 8. Normalize metrics
# ----------------------------

def normalize(series):
    if series.max() == series.min():
        return pd.Series([0.5] * len(series), index=series.index)
    return (series - series.min()) / (series.max() - series.min())

course_summary["enrollment_score"] = normalize(course_summary["students_cour
course_summary["skill_gain_score"] = normalize(course_summary["avg_progress"
course_summary["employment_score"] = normalize(course_summary["employment_ra
course_summary["future_score"] = course_summary["future_demand_score"] / 10
course_summary["equity_score"] = course_summary["female_share"].fillna(0)

# ----------------------------
# 9. Course optimization score
# ----------------------------
# Editable weights

course_summary["optimization_score"] = (
    0.30 * course_summary["employment_score"] +
    0.25 * course_summary["skill_gain_score"] +
    0.20 * course_summary["future_score"] +
    0.15 * course_summary["enrollment_score"] +
    0.10 * course_summary["equity_score"]
)
```

```
course_summary["optimization_score"] = course_summary["optimization_score"]

# ------------------------------
# 10. Recommendation logic
# ------------------------------

def recommend(row):
    if row["optimization_score"] >= 75:
        return "Expand"
    elif row["optimization_score"] >= 55:
        return "Maintain / improve"
    elif row["future_demand_score"] >= 8 and row["students_count"] < 30:
        return "Test again / market better"
    else:
        return "Reevaluate"

course_summary["recommendation"] = course_summary.apply(recommend, axis=1)

course_summary = course_summary.sort_values("optimization_score", ascending=

# ------------------------------
# 11. Save output
# ------------------------------

course_summary.to_csv("ilmhona_course_portfolio_optimization.csv", index=Fal
students.to_csv("ilmhona_cleaned_student_outcomes.csv", index=False)

print("Course Portfolio Optimization Results:")
print(course_summary[[
    "course",
    "students_count",
    "avg_progress",
    "employment_rate",
    "female_share",
    "future_demand_score",
    "optimization_score",
    "recommendation"
]].to_string(index=False))

# ------------------------------
# 12. Chart
# ------------------------------

plt.figure(figsize=(10, 6))

plt.barh(
    course_summary["course"],
    course_summary["optimization_score"]
)

plt.xlabel("Optimization Score")
plt.ylabel("Course")
plt.title("Ilmhona Course Portfolio Optimization Score")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig("ilmhona_course_optimization_scores.png")
```

```
plt.show()

# Example cost assumptions — edit course names to match your output exactly
course_costs_tjs = {
    "Digital Literacy": 500,
    "ICT": 500,
    "Basics of Programming": 500,
    "Programming": 600,
    "Java Script": 550,
    "Web design": 500,
    "Graphic Design": 700,
    "SMM": 700
}

course_summary["cost_per_student_tjs"] = course_summary["course"].map(course

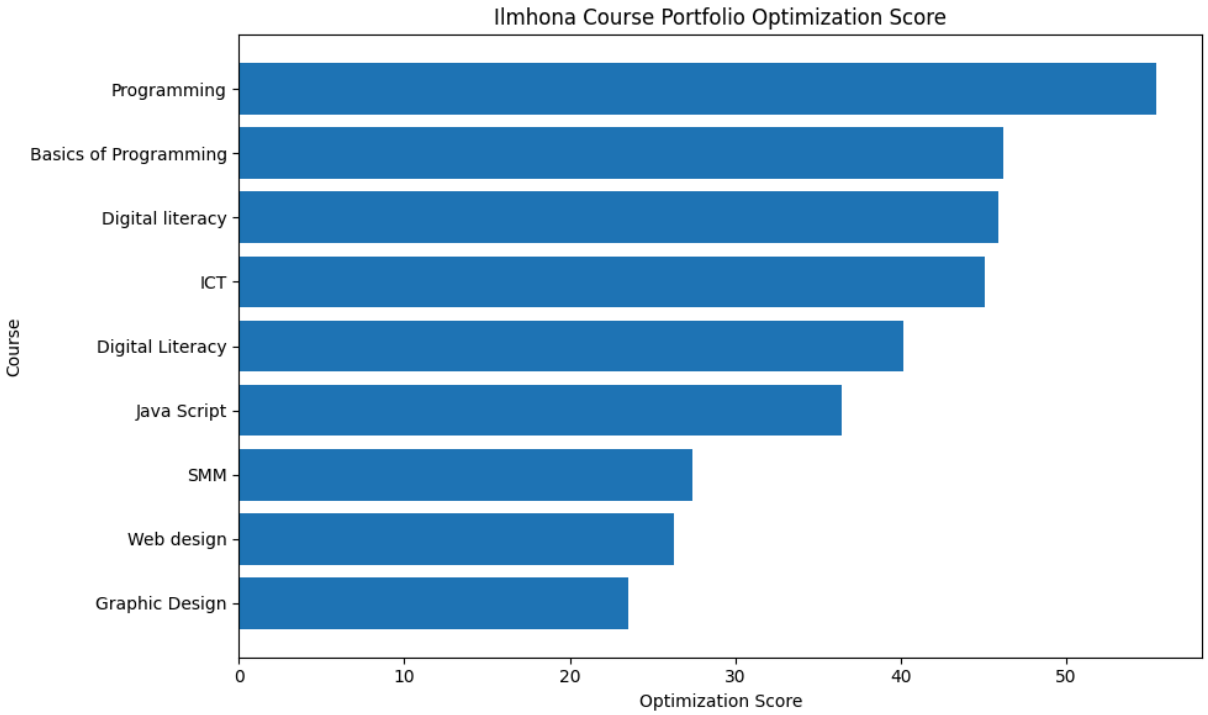
course_summary["total_course_cost_tjs"] = (
    course_summary["students_count"] * course_summary["cost_per_student_tjs"
)

course_summary["cost_per_employed_graduate_tjs"] = (
    course_summary["total_course_cost_tjs"] /
    course_summary["employment_count"].replace(0, np.nan)
)

# Lower cost per employed graduate = better
course_summary["cost_efficiency_score"] = 1 - normalize(
    course_summary["cost_per_employed_graduate_tjs"].fillna(
        course_summary["cost_per_employed_graduate_tjs"].max()
    )
)

# Updated optimization score
course_summary["optimization_score"] = (
    0.25 * course_summary["employment_score"] +
    0.20 * course_summary["skill_gain_score"] +
    0.20 * course_summary["future_score"] +
    0.15 * course_summary["enrollment_score"] +
    0.10 * course_summary["equity_score"] +
    0.10 * course_summary["cost_efficiency_score"]
) * 100
```

```
Course Portfolio Optimization Results:
              course  students_count  avg_progress  employment_rate  female
_share  future_demand_score  optimization_score             recommendation
          Programming              29     39.655172         1.000000      0.
689655                    9           55.448022         Maintain / improve
Basics of Programming              12     72.500000         0.000000      0.
500000                    8           46.176471 Test again / market better
     Digital literacy             404     64.792746         0.014851      0.
148515                    8           45.887809                Reevaluate
                 ICT             684     50.492690         0.020468      0.
526316                    8           45.126222                Reevaluate
     Digital Literacy             291     56.646048         0.003436      0.
481100                    8           40.177659                Reevaluate
          Java Script               7     60.000000         0.000000      0.
285714                    9           36.408884 Test again / market better
                 SMM              34     50.882353         0.147059      0.
176471                    6           27.383858                Reevaluate
           Web design               4     46.000000         0.000000      0.
750000                    7           26.329396                Reevaluate
       Graphic Design             139     46.258993         0.021583      0.
287770                    6           23.529651                Reevaluate
```




In [6]:

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# 1. Load files
# ----------------------------

regional_files = {
   "Dushanbe": "DB Impact source hubs 2025- 2026 - Dushanbe ISH.csv",
    "Shugnan": "DB Impact source hubs 2025- 2026 - Shugnan ISH.csv",
    "Bokhtar": "DB Impact source hubs 2025- 2026 - Bokhtar ISH.csv",
```

```
    "Panjikent": "DB Impact source hubs 2025- 2026 - Panjikent ISH.csv"
}

employment_file = "DB Impact source hubs 2025- 2026 - Employment.csv"

all_students = []

for region, file in regional_files.items():
    temp = pd.read_csv(file)
    temp["region_clean"] = region
    all_students.append(temp)

students = pd.concat(all_students, ignore_index=True)
employment = pd.read_csv(employment_file)

# ----------------------------
# 2. Clean column names
# ----------------------------

def clean_columns(df):
    df.columns = (
        df.columns
        .str.strip()
        .str.lower()
        .str.replace("\n", " ", regex=False)
        .str.replace("  ", " ", regex=False)
    )
    return df

students = clean_columns(students)
employment = clean_columns(employment)

print("Student columns:")
print(students.columns.tolist())

print("\nEmployment columns:")
print(employment.columns.tolist())

# ----------------------------
# 3. Rename columns
# ----------------------------

students = students.rename(columns={
    "full name": "student_name",
    "gender пол": "gender",
    "age возраст": "age",
    "courses received/ название курса": "course",
    "pre-test": "pre_test",
    "post-test": "post_test",
    "progress made": "progress",
    "amount earned (tjs)": "amount_earned"
})

employment = employment.rename(columns={
    "full name": "student_name",
    "courses received/ название курса": "course",
```

```
    "province name": "region",
    "organization employed": "organization_employed",
    "amount earned (tjs)": "employment_amount_earned"
})

# ----------------------------
# 4. Clean key fields
# ----------------------------

students["student_name"] = students["student_name"].astype(str).str.lower().
employment["student_name"] = employment["student_name"].astype(str).str.lowe

students["course"] = students["course"].astype(str).str.strip()
employment["course"] = employment["course"].astype(str).str.strip()

# Standardize course names
course_mapping = {
    "Digital literacy": "Digital Literacy",
    "digital literacy": "Digital Literacy",
    "ICT": "Digital Literacy",
    "ict": "Digital Literacy",
    "Java Script": "JavaScript",
    "java script": "JavaScript",
    "Web design": "Web Design",
    "web design": "Web Design"
}

students["course"] = students["course"].replace(course_mapping)
employment["course"] = employment["course"].replace(course_mapping)

# Remove blank course rows
students = students[students["course"].notna()]
students = students[students["course"] != "nan"]

# Convert numeric columns
for col in ["age", "pre_test", "post_test", "progress", "amount_earned"]:
    if col in students.columns:
        students[col] = (
            students[col]
            .astype(str)
            .str.replace("%", "", regex=False)
            .str.replace(",", "", regex=False)
            .str.strip()
        )
        students[col] = pd.to_numeric(students[col], errors="coerce")

if "employment_amount_earned" in employment.columns:
    employment["employment_amount_earned"] = (
        employment["employment_amount_earned"]
        .astype(str)
        .str.replace("%", "", regex=False)
        .str.replace(",", "", regex=False)
        .str.strip()
    )
    employment["employment_amount_earned"] = pd.to_numeric(
        employment["employment_amount_earned"],
```

```
        errors="coerce"
    )

# ----------------------------
# 5. Add employment flag
# ----------------------------

employment["employed_flag"] = 1

students = students.merge(
    employment[["student_name", "employed_flag"]],
    on="student_name",
    how="left"
)

students["employed_flag"] = students["employed_flag"].fillna(0)

# If earnings are missing in student file, use employment file if available
if "employment_amount_earned" in employment.columns:
    earnings_lookup = employment[["student_name", "employment_amount_earned"
    students = students.merge(earnings_lookup, on="student_name", how="left"

    if "amount_earned" not in students.columns:
        students["amount_earned"] = students["employment_amount_earned"]
    else:
        students["amount_earned"] = students["amount_earned"].fillna(
            students["employment_amount_earned"]
        )

# ----------------------------
# 6. Gender cleaning
# ----------------------------

students["gender_clean"] = students["gender"].astype(str).str.lower().str.st

students["female_flag"] = students["gender_clean"].str.contains(
    "female|f|жен|зан",
    case=False,
    na=False
).astype(int)

# ----------------------------
# 7. Executive KPIs
# ----------------------------

total_students = len(students)
total_employed = students["employed_flag"].sum()
employment_rate = students["employed_flag"].mean() * 100
avg_earnings = students["amount_earned"].mean()
avg_progress = students["progress"].mean()
female_share = students["female_flag"].mean() * 100

kpi_summary = pd.DataFrame({
    "Metric": [
        "Total students",
        "Employed graduates",
```

```
        "Employment rate (%)",
        "Average earnings (TJS)",
        "Average skill progress",
        "Female participation (%)"
    ],
    "Value": [
        total_students,
        total_employed,
        employment_rate,
        avg_earnings,
        avg_progress,
        female_share
    ]
})

print("\nExecutive KPIs:")
print(kpi_summary.to_string(index=False))

# ------------------------------
# 8. Course outcomes
# ------------------------------

course_summary = students.groupby("course").agg(
    students_count=("student_name", "count"),
    employed_count=("employed_flag", "sum"),
    employment_rate=("employed_flag", "mean"),
    avg_earnings=("amount_earned", "mean"),
    avg_pre_test=("pre_test", "mean"),
    avg_post_test=("post_test", "mean"),
    avg_progress=("progress", "mean"),
    female_share=("female_flag", "mean")
).reset_index()

course_summary["employment_rate"] *= 100
course_summary["female_share"] *= 100

course_summary = course_summary.sort_values(
    "employment_rate",
    ascending=False
)

print("\nCourse Outcomes:")
print(course_summary.to_string(index=False))

# ------------------------------
# 9. Regional outcomes
# ------------------------------

region_summary = students.groupby("region_clean").agg(
    students_count=("student_name", "count"),
    employed_count=("employed_flag", "sum"),
    employment_rate=("employed_flag", "mean"),
    avg_earnings=("amount_earned", "mean"),
    avg_progress=("progress", "mean"),
    female_share=("female_flag", "mean")
).reset_index()
```

```
region_summary["employment_rate"] *= 100
region_summary["female_share"] *= 100

print("\nRegional Outcomes:")
print(region_summary.to_string(index=False))

# ----------------------------
# 10. Gender outcomes
# ----------------------------

gender_summary = students.groupby("gender").agg(
    students_count=("student_name", "count"),
    employed_count=("employed_flag", "sum"),
    employment_rate=("employed_flag", "mean"),
    avg_earnings=("amount_earned", "mean"),
    avg_progress=("progress", "mean")
).reset_index()

gender_summary["employment_rate"] *= 100

print("\nGender Outcomes:")
print(gender_summary.to_string(index=False))

# ----------------------------
# 11. Skill-to-employment pipeline
# ----------------------------

pipeline = students.groupby("course").agg(
    students_count=("student_name", "count"),
    avg_progress=("progress", "mean"),
    employment_rate=("employed_flag", "mean"),
    avg_earnings=("amount_earned", "mean")
).reset_index()

pipeline["employment_rate"] *= 100

# Expected earnings per student
pipeline["expected_earnings_per_student"] = (
    pipeline["avg_earnings"].fillna(0) *
    pipeline["employment_rate"].fillna(0) / 100
)

print("\nSkill-to-Employment Pipeline:")
print(pipeline.to_string(index=False))

# ----------------------------
# 12. Save dashboard tables
# ----------------------------

students.to_csv("ilmhona_cleaned_student_employment_data.csv", index=False)
kpi_summary.to_csv("dashboard_kpi_summary.csv", index=False)
course_summary.to_csv("dashboard_course_outcomes.csv", index=False)
region_summary.to_csv("dashboard_region_outcomes.csv", index=False)
gender_summary.to_csv("dashboard_gender_outcomes.csv", index=False)
pipeline.to_csv("dashboard_skill_employment_pipeline.csv", index=False)
```

```
# ----------------------------
# 13. Charts
# ----------------------------

# Employment by course
plt.figure(figsize=(10, 6))
plt.barh(course_summary["course"], course_summary["employment_rate"])
plt.xlabel("Employment Rate (%)")
plt.ylabel("Course")
plt.title("Employment Rate by Course")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig("employment_rate_by_course.png")
plt.show()

# Average earnings by course
earnings_chart = course_summary.dropna(subset=["avg_earnings"]).copy()

plt.figure(figsize=(10, 6))
plt.barh(earnings_chart["course"], earnings_chart["avg_earnings"])
plt.xlabel("Average Earnings (TJS)")
plt.ylabel("Course")
plt.title("Average Earnings by Course")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig("average_earnings_by_course.png")
plt.show()

# Skill progress by course
progress_chart = course_summary.dropna(subset=["avg_progress"]).copy()
progress_chart = progress_chart.sort_values("avg_progress", ascending=False)

plt.figure(figsize=(10, 6))
plt.barh(progress_chart["course"], progress_chart["avg_progress"])
plt.xlabel("Average Skill Progress")
plt.ylabel("Course")
plt.title("Average Skill Progress by Course")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig("skill_progress_by_course.png")
plt.show()

# Employment by region
plt.figure(figsize=(8, 5))
plt.bar(region_summary["region_clean"], region_summary["employment_rate"])
plt.xlabel("Region")
plt.ylabel("Employment Rate (%)")
plt.title("Employment Rate by Region")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig("employment_rate_by_region.png")
plt.show()

# Skill progress vs employment scatter
plt.figure(figsize=(9, 6))
```

```
plt.scatter(
    pipeline["avg_progress"],
    pipeline["employment_rate"],
    s=pipeline["students_count"] * 3
)

for _, row in pipeline.iterrows():
    plt.text(
        row["avg_progress"],
        row["employment_rate"],
        row["course"],
        fontsize=9
    )

plt.xlabel("Average Skill Progress")
plt.ylabel("Employment Rate (%)")
plt.title("Skill Progress vs Employment Rate by Course")
plt.tight_layout()
plt.savefig("skill_progress_vs_employment.png")
plt.show()

# ----------------------------
# 14. Simple interpretation
# ----------------------------

print("\nKey Takeaways:")

top_employment = course_summary.sort_values(
    "employment_rate",
    ascending=False
).head(3)

top_progress = course_summary.sort_values(
    "avg_progress",
    ascending=False
).head(3)

largest_courses = course_summary.sort_values(
    "students_count",
    ascending=False
).head(3)

print("\nTop courses by employment rate:")
print(top_employment[["course", "employment_rate", "students_count"]].to_str

print("\nTop courses by skill progress:")
print(top_progress[["course", "avg_progress", "students_count"]].to_string(i

print("\nLargest courses by enrollment:")
print(largest_courses[["course", "students_count"]].to_string(index=False))

print("\nFiles saved:")
print("ilmhona_cleaned_student_employment_data.csv")
print("dashboard_kpi_summary.csv")
print("dashboard_course_outcomes.csv")
print("dashboard_region_outcomes.csv")
```

```
print("dashboard_gender_outcomes.csv")
print("dashboard_skill_employment_pipeline.csv")
print("employment_rate_by_course.png")
print("average_earnings_by_course.png")
print("skill_progress_by_course.png")
print("employment_rate_by_region.png")
print("skill_progress_vs_employment.png")
```

```
Student columns:
['unnamed: 0', 'organization', 'province name', 'full name', 'birth date дат
а рождения', 'age возраст', 'gender пол', 'courses received/ название курс
а', 'date of first attendance дата начала курса', 'date of course completion
дата завершения', 'pre-test', 'post-test', 'progress made', 'phone number',
'location type/тип местности', 'neet status', 'basic ict skills (ms office e
tc.)', 'amount earned (tjs)', 'employer - local or international трудойстрой
ство', 'name of organization название организации, если трудоустроили', 'reg
ion_clean', '№', 'phone number']

Employment columns:
['№', 'full name', 'age', 'gender', 'organization', 'province name', 'course
s received/ название курса', 'column 1', 'organization employed', 'phone num
ber']

Executive KPIs:
                  Metric       Value
          Total students 1604.000000
      Employed graduates   58.000000
     Employment rate (%)    3.615960
  Average earnings (TJS)         NaN
  Average skill progress   53.345298
Female participation (%)   39.713217

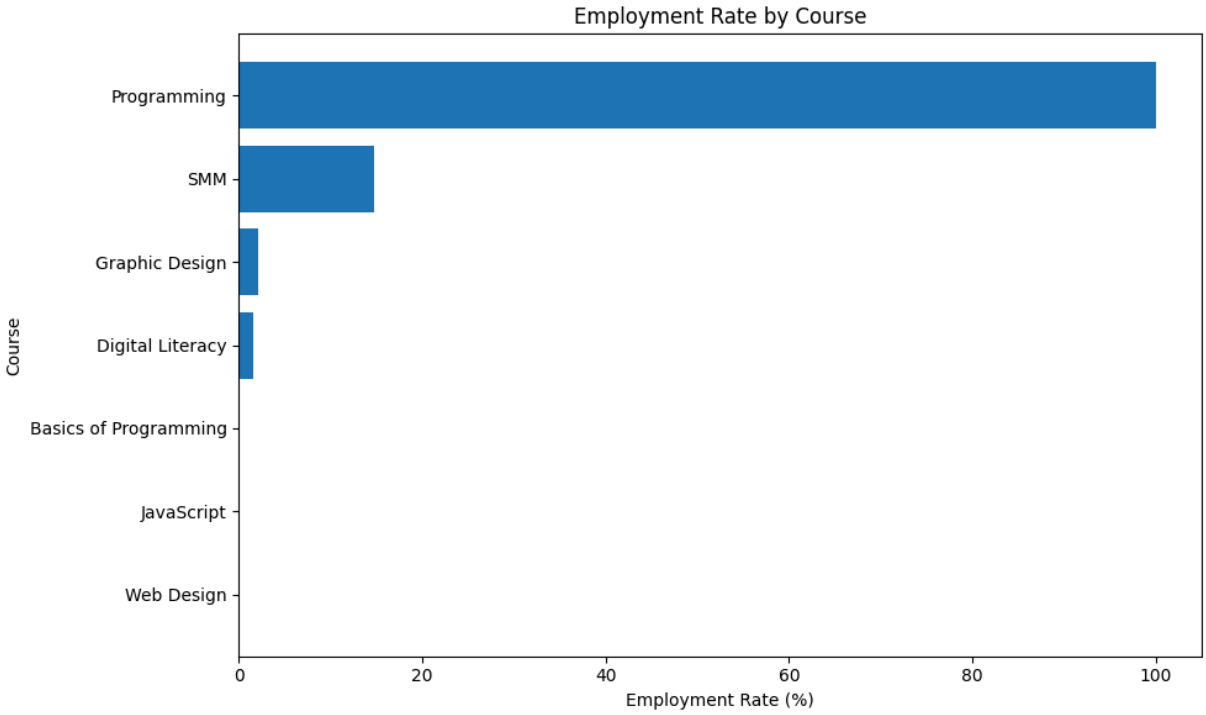
Course Outcomes:
               course  students_count  employed_count  employment_rate  avg_
earnings  avg_pre_test  avg_post_test  avg_progress  female_share
          Programming              29            29.0       100.000000
NaN     33.448276      73.103448     39.655172     68.965517
                 SMM              34             5.0        14.705882
NaN     44.558824      75.735294     50.882353     17.647059
       Graphic Design             139             3.0         2.158273
NaN     29.172662      60.467626     46.258993     28.776978
     Digital Literacy            1379            21.0         1.522843
NaN     22.317637      68.534247     54.388699     40.609137
Basics of Programming              12             0.0         0.000000
NaN     32.083333      40.416667     72.500000     50.000000
           JavaScript               7             0.0         0.000000
NaN     16.428571      76.428571     60.000000     28.571429
           Web Design               4             0.0         0.000000
NaN     36.000000      82.000000     46.000000     75.000000

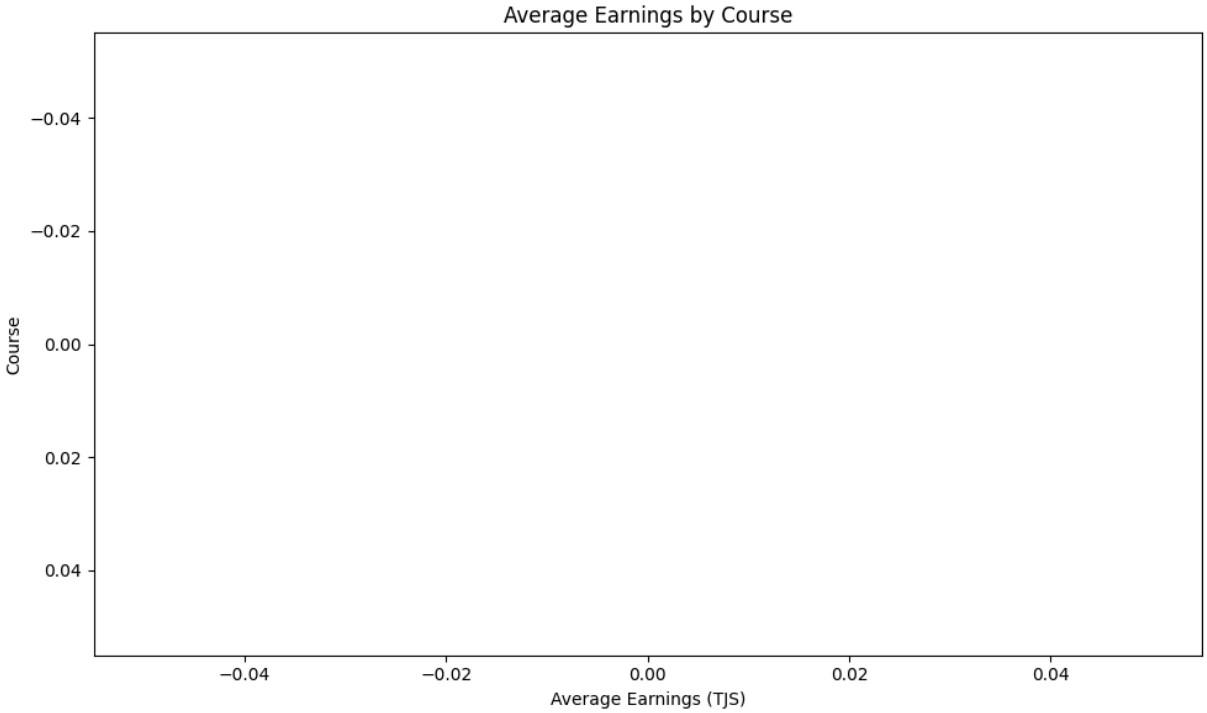
Regional Outcomes:
region_clean  students_count  employed_count  employment_rate  avg_earnings
avg_progress  female_share
     Bokhtar             476             9.0         1.890756           NaN
53.600840     40.336134
    Dushanbe             411             6.0         1.459854           NaN
64.625000     15.085158
   Panjikent             250            37.0        14.800000           NaN
38.368000     61.200000
     Shugnan             467             6.0         1.284797           NaN
56.271949     49.250535

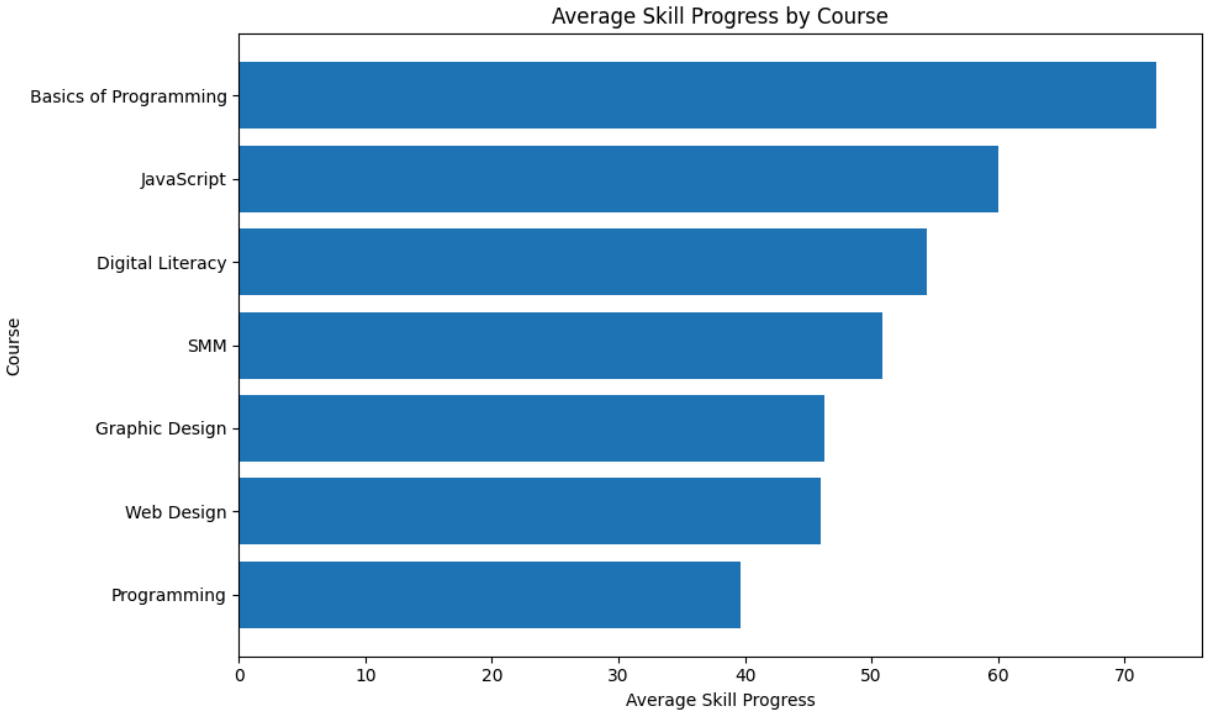
Gender Outcomes:
gender  students_count  employed_count  employment_rate  avg_earnings  avg_p
```

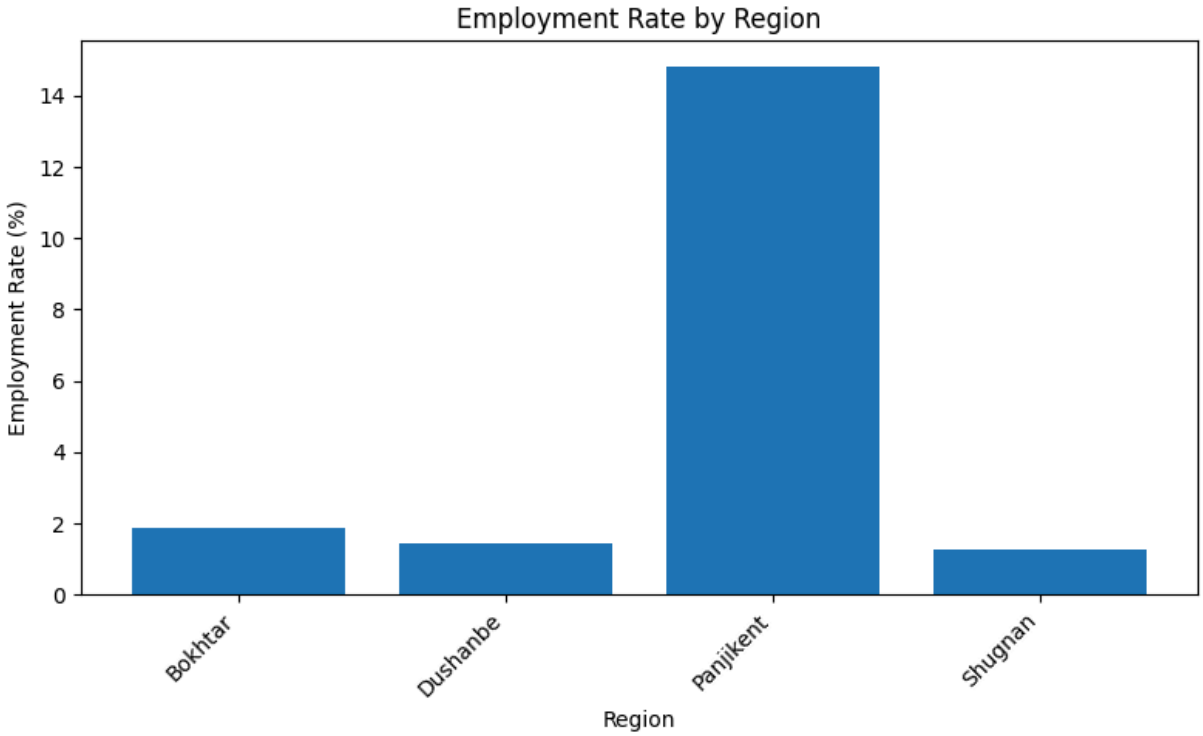
```
rogress
Female             637            30.0         4.709576           NaN      5
0.612416
  Male             967            28.0         2.895553           NaN      5
5.388959

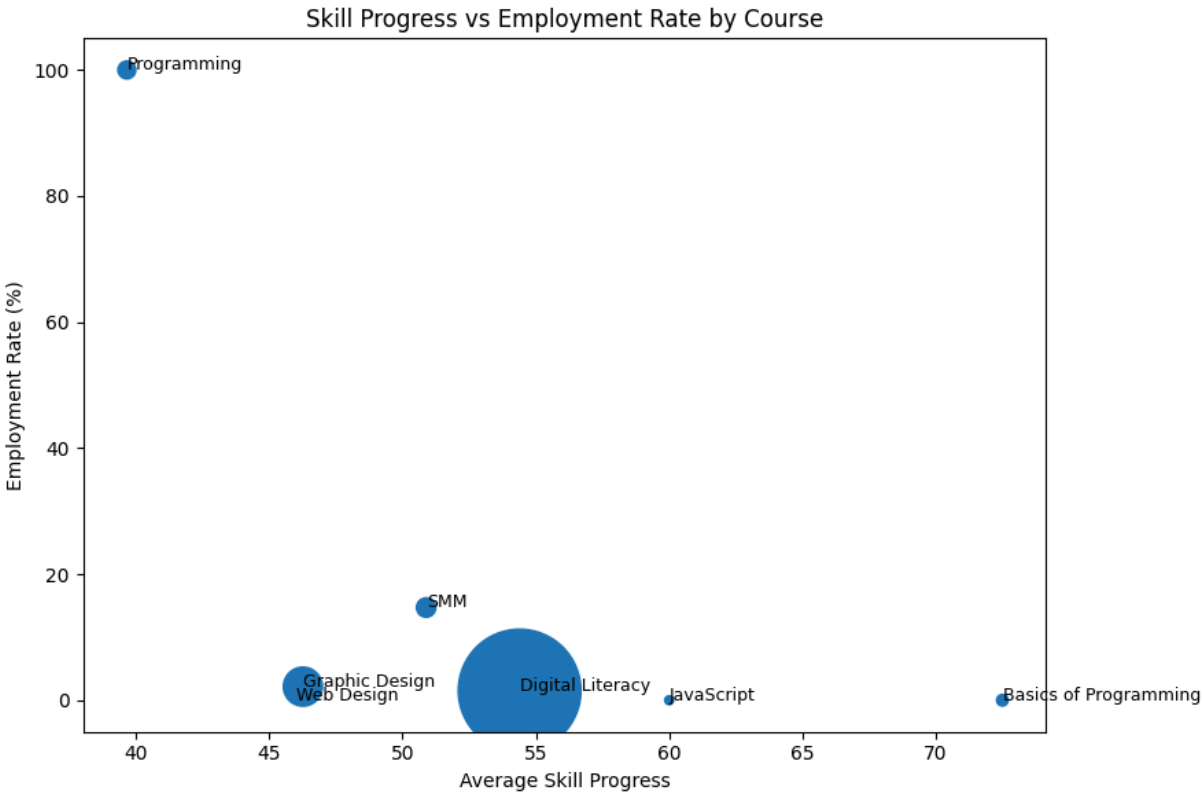
Skill-to-Employment Pipeline:
               course  students_count  avg_progress  employment_rate  avg_ea
rnings  expected_earnings_per_student
Basics of Programming              12     72.500000         0.000000
NaN                           0.0
     Digital Literacy            1379     54.388699         1.522843
NaN                           0.0
       Graphic Design             139     46.258993         2.158273
NaN                           0.0
           JavaScript               7     60.000000         0.000000
NaN                           0.0
          Programming              29     39.655172       100.000000
NaN                           0.0
                 SMM               34     50.882353        14.705882
NaN                           0.0
          Web Design               4     46.000000         0.000000
NaN                           0.0
```




Average Earnings by Course
Course
−0.04
−0.02
0.00
0.02
0.04
−0.04
−0.02
0.00
0.02
0.04
Average Earnings (TJS)


Average Skill Progress by Course
Course
Basics of Programming
JavaScript
Digital Literacy
SMM
Graphic Design
Web Design
Programming
0
10
20
30
40
50
60
70
Average Skill Progress


Employment Rate by Region
Employment Rate (%)
0
2
4
6
8
10
12
14
Bokhtar
Dushanbe
Panjikent
Shugnan
Region


Skill Progress vs Employment Rate by Course
Employment Rate (%)
0
20
40
60
80
100
Programming
SMM
Graphic Design
Web Design
Digital Literacy
JavaScript
Basics of Programming
40
45
50
55
60
65
70
Average Skill Progress

```
Key Takeaways:

Top courses by employment rate:
        course  employment_rate  students_count
   Programming       100.000000              29
           SMM        14.705882              34
Graphic Design         2.158273             139

Top courses by skill progress:
               course  avg_progress  students_count
Basics of Programming     72.500000              12
           JavaScript     60.000000               7
     Digital Literacy     54.388699            1379

Largest courses by enrollment:
          course  students_count
Digital Literacy            1379
  Graphic Design             139
             SMM              34

Files saved:
ilmhona_cleaned_student_employment_data.csv
dashboard_kpi_summary.csv
dashboard_course_outcomes.csv
dashboard_region_outcomes.csv
dashboard_gender_outcomes.csv
dashboard_skill_employment_pipeline.csv
employment_rate_by_course.png
average_earnings_by_course.png
skill_progress_by_course.png
employment_rate_by_region.png
skill_progress_vs_employment.png
```

In [ ]: